

Arquitectura de Soluciones — Banca por Internet

Documento Técnico | Entidad Financiera BP

Autor: Santiago Ricardo Del Salto Espinosa — Lead Software Architect

Fecha: Septiembre 2026 | **Versión:** 7.0

1. Resumen Ejecutivo y Visión de Negocio

El presente documento formaliza el diseño técnico de la plataforma de Banca por Internet de la entidad financiera **BP**. La solución aborda los requerimientos de consulta de movimientos, transferencias propias e interbancarias, onboarding biométrico y auditoría inmutable, bajo estrictos estándares de la industria bancaria y principios de arquitectura orientada a dominios (DDD) y sistemas distribuidos resilientes.

2. Análisis Técnico de Decisiones Arquitectónicas (Trade-Offs)

Como parte del gobierno técnico, cada componente ha sido seleccionado ponderando rendimiento, mantenibilidad a largo plazo y desacoplamiento:

Dominio / Componente	Alternativas Evaluadas	Decisión Adoptada	Justificación Técnica y de Negocio
Frontend Web (SPA)	React, Vue.js, Angular	Angular (TypeScript)	Estructura modular estricta e inyección de dependencias nativa ideal para dominios financieros complejos. Uso de RxJS para gestión reactiva de flujos asíncronos en tiempo real.

Dominio / Componente	Alternativas Evaluadas	Decisión Adoptada	Justificación Técnica y de Negocio
App Móvil	React Native, Native (Swift/Kotlin), Flutter	Flutter (Dart)	Rendimiento gráfico superior (120 FPS con motor Skia/Impeller) indispensable para la fluidez en el flujo de liveness check biométrico. Compilación AOT a código máquina nativo.
API Gateway	Kong, Custom NGINX, AWS/Azure APIM	AWS API / Azure API Managed	Offloading nativo de políticas de seguridad (JWT validation, WAF, throttling de IP) protegiendo el perímetro de backend y gestionando versionado de contratos.
Backend Core	Node.js (NestJS), Quarkus, Spring Boot	Spring Boot + WebFlux (Java 21)	Modelo de concurrencia reactiva no bloqueante idóneo para soportar picos altos de consulta de movimientos bancarios con mínimo consumo de recursos.
Bus de Mensajería	RabbitMQ, AWS SQS, Apache Kafka	Apache Kafka	Capacidad nativa de persistencia en disco (log-based messaging) permitiendo auditoría

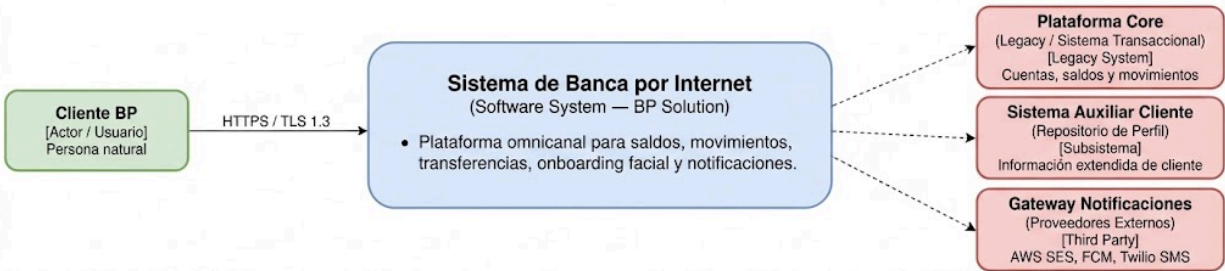
Dominio / Componente	Alternativas Evaluadas	Decisión Adoptada	Justificación Técnica y de Negocio
			forense y event replay ante fallos, con particionamiento horizontal masivo.

3. Modelos C4 de Arquitectura

Modelo C4 — Nivel 1: Diagrama de Contexto del Sistema (Enterprise Landscape)

- **Actores y Sistemas Principales:**
 - **Cliente BP:** Actor / Usuario (Persona natural o jurídica).
 - **Sistema de Banca por Internet (Software System — BP Solution):**
Plataforma omnicanal para saldos, movimientos, transferencias, onboarding facial y notificaciones.
 - **Plataforma Core:** Sistema transaccional legacy (Cuentas, saldos y movimientos).
 - **Sistema Auxiliar Cliente:** Repositorio de perfil extendido.
 - **Gateway Notificaciones:** Proveedores externos (AWS SES, FCM, Twilio SMS).
- **Protocolos de Conectividad:** Canales cifrados HTTPS / TLS 1.3 entre el cliente y el API Gateway; integración segura con sistemas core mediante APIs REST asíncronas.

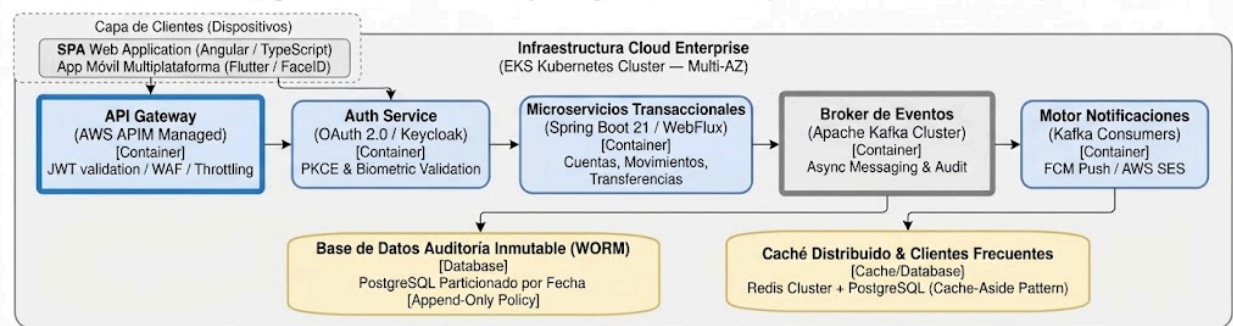
Modelo C4 — Nivel 1: Diagrama de Contexto del Sistema (Enterprise Landscape)



Modelo C4 — Nivel 2: Diagrama de Contenedores y Componentes Cloud

- **Capa de Clientes:** SPA Web [Angular] y App Móvil [Flutter / FaceID / SecureStore].
- **Infraestructura Cloud (EKS Kubernetes Cluster - Multi-AZ):**
 - **API Gateway:** AWS APIM / WAF / Throttling.
 - **Auth Service:** Keycloak / OAuth 2.0 / PKCE & Biometric Validation.
 - **Microservicios Transaccionales:** Spring Boot 21 / WebFlux (Cuentas, Movimientos, Transferencias).
 - **Broker de Eventos:** Apache Kafka Cluster.
 - **Motor de Notificaciones:** Kafka Consumers (FCM Push / AWS SES).
- **Capa de Persistencia Especializada:**
 - **Base de Datos Auditoría WORM:** PostgreSQL Append-Only Particionado por fecha (política estricta sin permisos de **UPDATE** ni **DELETE**).
 - **Caché & Clientes Frecuentes:** Redis Cluster + PostgreSQL (Patrón Cache-Aside).

Modelo C4 — Nivel 2: Diagrama de Contenedores y Componentes Cloud (Kubernetes & AWS/Azure)



4. Especificaciones Técnicas y de Seguridad

Autenticación y Onboarding (OAuth 2.0 / PKCE)

- **Flujo Authorization Code con PKCE:** Requerido para clientes públicos (SPA y App Móvil), evitando ataques de interceptación de códigos mediante parámetros criptográficos dinámicos (**code_verifier** y **code_challenge** con SHA-256).
- **Onboarding Biométrico:** Incorpora reconocimiento facial y detección de vivacidad activa (*liveness check*) integrados con pasarelas biométricas certificadas. Las credenciales de sesión local se protegen en el *Secure Enclave* / *KeyChain* del dispositivo.

Patrones de Diseño y Rendimiento

- **Patrón Audit Logging (WORM / Append-Only):** Publicación de eventos estructurados hacia Apache Kafka y persistencia en PostgreSQL particionado con roles de base de datos restringidos (sin comandos **UPDATE** ni **DELETE**), asegurando inmutabilidad forense ante auditorías regulatorias.
- **Patrón Cache-Aside con Redis Cluster:** Optimización de consultas frecuentes de saldos y movimientos mediante Redis Cluster (TTL configurable) con repoblamiento asíncrono en caso de *cache miss*.

Consideraciones Cloud, HA, DR y Seguridad

- **Alta Disponibilidad (HA):** Despliegue en Kubernetes (EKS / AKS) multi-zona (Multi-AZ) con balanceadores de carga globales y Auto-Scaling horizontal (HPA).
- **Recuperación ante Desastres (DR):** Estrategia *Active-Passive* con replicación asíncrona multi-región (RPO < 1 min, RTO < 15 min).
- **Cifrado y Cumplimiento:** Cifrado de extremo a extremo con TLS 1.3 en tránsito y AES-256 en reposo. Gestión centralizada de secretos con AWS Secrets Manager / Azure Key Vault.